

Information Security : TP Breaking the Classics

Riad Benbrahim

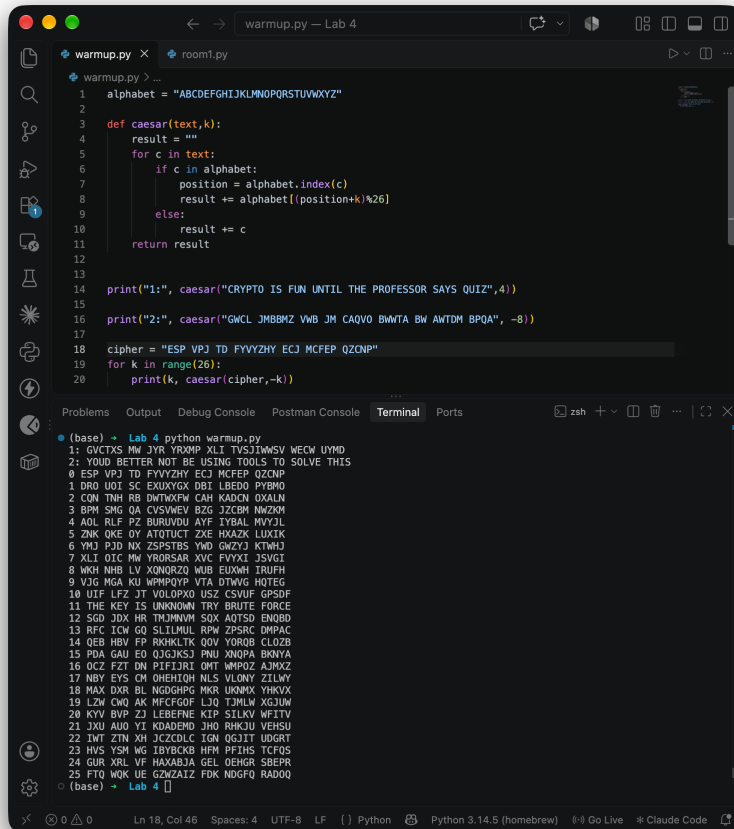
October 6, 2026

Contents

1	Warm up	2
2	Room 1	2
2.1	1. and 2.	2
2.2	3. and 4.	3
3	Room 2	5

1 Warm up

Encrypting the first message with $K = 4$ gives GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD. Decrypting the second with $K = 8$ gives YOU'D BETTER NOT BE USING TOOLS TO SOLVE THIS. Brute force recovers THE KEY IS UNKNOWN TRY BRUTE FORCE with key $K = 11$.



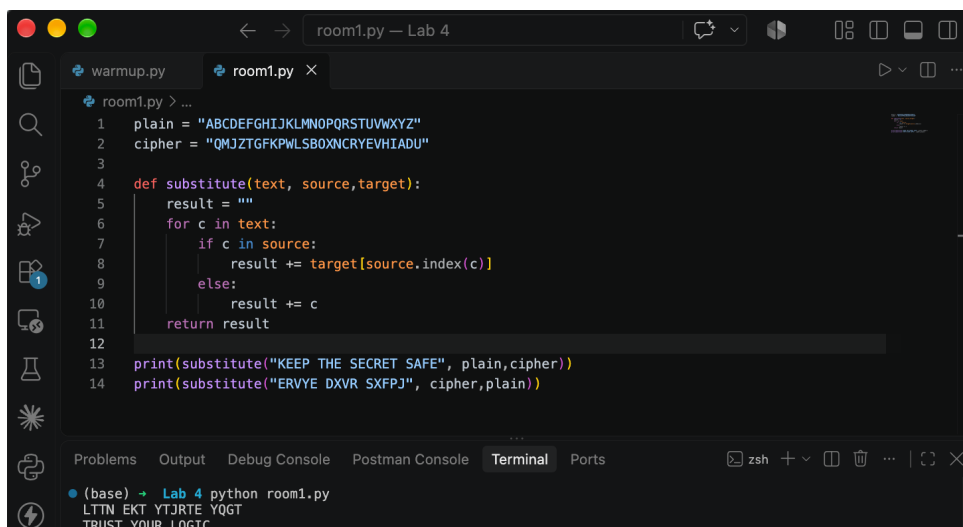
```
warmup.py X room1.py
warmup.py > ...
1 alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
2
3 def caesar(text,k):
4     result = ""
5     for c in text:
6         if c in alphabet:
7             position = alphabet.index(c)
8             result += alphabet[(position+k)%26]
9         else:
10            result += c
11    return result
12
13 print("1:", caesar("CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ",4))
14
15 print("2:", caesar("GWCL JMBBMZ VMB JM CAQVO BMTA BW AWTDM BPQA", -8))
16
17 cipher = "ESP VPJ TD FYVYZHY ECJ MCFEP QZONP"
18 for k in range(26):
19     print(k, caesar(cipher,-k))
```

Problems Output Debug Console Postman Console Terminal Ports

```
(base) → Lab 4 python warmup.py
1: GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD
2: YOU'D BETTER NOT BE USING TOOLS TO SOLVE THIS
3: ESP VPJ TD FYVYZHY ECJ MCFEP QZONP
4: DRD UOI SC EXUYGX DBI LBEO PYBMO
5: CON TNH RB DWTWKF CAH KADCN OXALN
6: BPM SMG QA CYSWVEV BZG JZCBM NWZKM
7: AOL RLF PZ BURUVDU AYE IYBAL WYVJL
8: ZWK QKE OY ATQTUET ZAE HBAZX LUXIK
9: YMJ PJD NX ZSPSTBS YMD GWZYJ KTMHJ
10: XLI OIC MW YRORSAR XVC FVYXI JSVGI
11: WKH NHB LV XQNRQZO WUB EUXMH IIRFH
12: YJG MGA KU WMPQVPT VTA DTWVG NQTEG
13: UIF LFZ JT VOLOPXO USZ CSVUF GPSDF
14: THE KEY IS UNKNOWN TRY BRUTE FORCE
15: SGD JDX HR TJMNWV SOX AOTSD ENQBD
16: RYC TCM GO SULLMUL RFW ZPSRC DMPAC
17: QEB HBV FP RKHKLTLC QOV YORQB CLOZB
18: PDA GAU EO QJGJKSJ PMU XNOPA BKNYA
19: OCZ FZI DN PIFJRI QNT WMPQZ AJMZX
20: NBY EYS CH OHEHQQ NLS VLQAY ZILWY
21: MAX DXR BL NGDHPG MKR UQOMX YHKVX
22: LZW QWQ AK MFCFGOF LJQ TJMLW XGJUM
23: KYV BVP ZJ LEBEFNE KIP SILKV WFITV
24: JXU AUG YI KQADQD JHD BKKJU VENSU
25: IWT ZTN XH JCZCBLC IGN QGJIT UDGRT
26: HVS YSM WG IBYBCKB HFM PFTHS TCFQS
27: GUR XRL VF HAVABJA GEL DEHGR SBEPH
28: FTO MOK UE GAKZALZ FDK NDCFQ RADOQ
```

2 Room 1

2.1 1. and 2.



```
warmup.py X room1.py X
room1.py > ...
1 plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
2 cipher = "QMJZTGFKPWLBSOXNCRYEVHIADU"
3
4 def substitute(text, source,target):
5     result = ""
6     for c in text:
7         if c in source:
8             result += target[source.index(c)]
9         else:
10            result += c
11    return result
12
13 print(substitute("KEEP THE SECRET SAFE", plain,cipher))
14 print(substitute("ERVYE DXVR SXFPJ", cipher,plain))
```

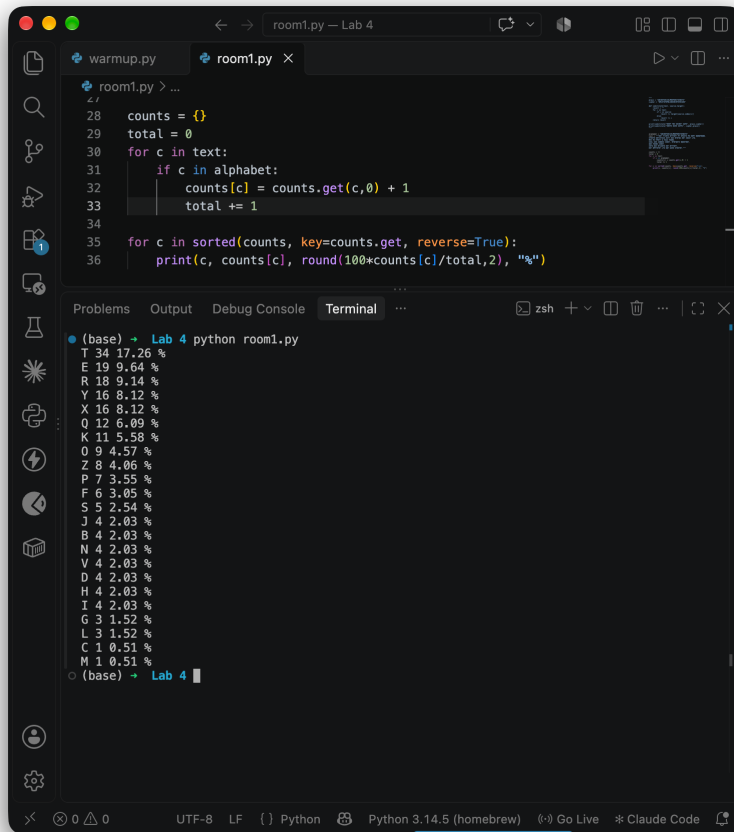
Problems Output Debug Console Postman Console Terminal Ports

```
(base) → Lab 4 python room1.py
LTTN EKT YJRTE YQGT
TRUST YOUR LOGIC
```

2.2 3. and 4.

Answer for the question :

An attacker counts ciphertext letters and compares their frequencies with typical English frequencies. For example, the most frequent symbol might represent E, suggesting one substitution pair. Word patterns then help confirm or correct these guesses and recover more of the key.

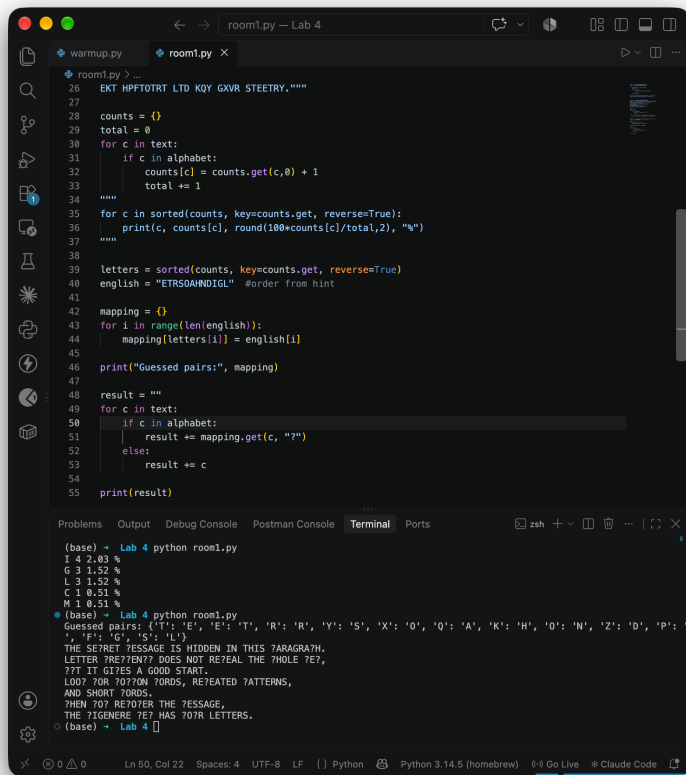


The image shows a VS Code editor window with a file named `room1.py` open. The script calculates the frequency of letters in a ciphertext. The terminal output shows the results, with 'T' being the most frequent letter at 17.26%.

```
room1.py > ...
28 counts = {}
29 total = 0
30 for c in text:
31     if c in alphabet:
32         counts[c] = counts.get(c,0) + 1
33         total += 1
34
35 for c in sorted(counts, key=counts.get, reverse=True):
36     print(c, counts[c], round(100*counts[c]/total,2), "%")
```

```
(base) → Lab 4 python room1.py
T 34 17.26 %
E 19 9.64 %
R 18 9.14 %
Y 16 8.12 %
X 16 8.12 %
Q 12 6.09 %
K 11 5.58 %
O 9 4.57 %
Z 8 4.06 %
P 7 3.55 %
F 6 3.05 %
S 5 2.54 %
J 4 2.03 %
B 4 2.03 %
N 4 2.03 %
V 4 2.03 %
D 4 2.03 %
H 4 2.03 %
I 4 2.03 %
G 3 1.52 %
L 3 1.52 %
C 1 0.51 %
M 1 0.51 %
(base) → Lab 4
```

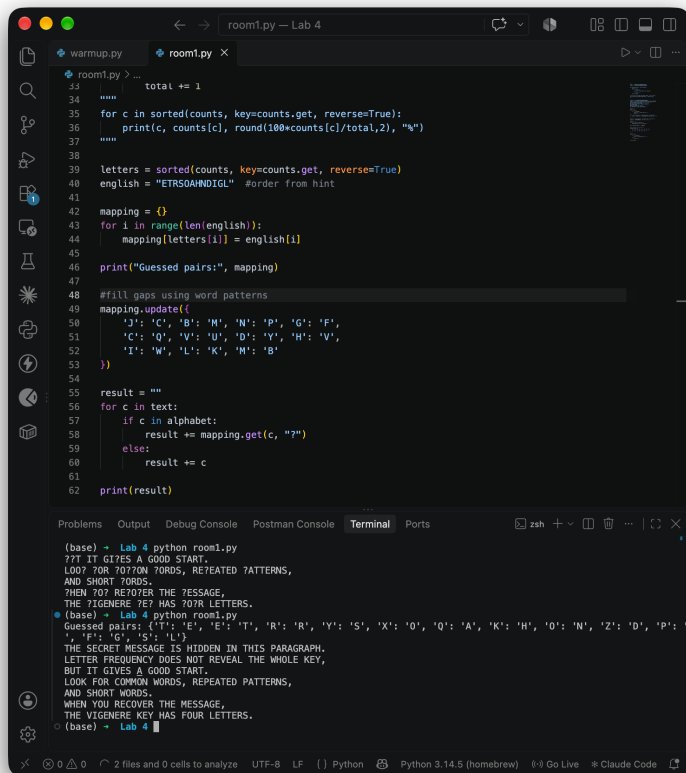
First attempt using hint :



```
room1.py
26 EKT HPFTOTRT LTD KQY GAVR STEETRY.""
27
28 counts = {}
29 total = 0
30 for c in text:
31     if c in alphabet:
32         counts[c] = counts.get(c,0) + 1
33         total += 1
34
35 for c in sorted(counts, key=counts.get, reverse=True):
36     print(c, counts[c], round(100*counts[c]/total,2), "%")
37
38
39 letters = sorted(counts, key=counts.get, reverse=True)
40 english = "ETRSOAHNDIGL" #order from hint
41
42 mapping = {}
43 for i in range(len(english)):
44     mapping[letters[i]] = english[i]
45
46 print("Guessed pairs:", mapping)
47
48 result = ""
49 for c in text:
50     if c in alphabet:
51         result += mapping.get(c, "?")
52     else:
53         result += c
54
55 print(result)
```

```
(base) ➔ Lab 4 python room1.py
I 4 2.03 %
G 3 1.52 %
L 3 1.52 %
C 1 0.51 %
M 1 0.51 %
(base) ➔ Lab 4 python room1.py
Guessed pairs: {'T': 'E', 'E': 'T', 'R': 'R', 'Y': 'S', 'X': 'O', 'O': 'A', 'K': 'H', 'O': 'N', 'Z': 'D', 'P': 'I', 'I': 'P', 'G': 'S', 'S': 'L'}
THE SECRET MESSAGE IS HIDDEN IN THIS PARAGRAPH.
LETTER FREQUENCY DOES NOT REVEAL THE WHOLE KEY,
BUT IT GIVES A GOOD START.
LOOK FOR COMMON WORDS, REPEATED PATTERNS,
AND SHORT WORDS.
WHEN YOU RECOVER THE MESSAGE,
THE VIGENERE KEY HAS FOUR LETTERS.
```

Update using word patterns :



```
room1.py
34
35
36 for c in sorted(counts, key=counts.get, reverse=True):
37     print(c, counts[c], round(100*counts[c]/total,2), "%")
38
39
40 letters = sorted(counts, key=counts.get, reverse=True)
41 english = "ETRSOAHNDIGL" #order from hint
42
43 mapping = {}
44 for i in range(len(english)):
45     mapping[letters[i]] = english[i]
46
47 print("Guessed pairs:", mapping)
48
49 #fill gaps using word patterns
50 mapping.update({
51     'J': 'C', 'B': 'M', 'N': 'P', 'G': 'F',
52     'C': 'O', 'V': 'U', 'D': 'Y', 'H': 'V',
53     'I': 'W', 'L': 'K', 'M': 'B'
54 })
55
56 result = ""
57 for c in text:
58     if c in alphabet:
59         result += mapping.get(c, "?")
60     else:
61         result += c
62
63 print(result)
```

```
(base) ➔ Lab 4 python room1.py
77T IT GIVES A GOOD START.
LOOK FOR COMMON WORDS, REPEATED PATTERNS,
AND SHORT WORDS.
WHEN YOU RECOVER THE MESSAGE,
THE VIGENERE KEY HAS FOUR LETTERS.
(base) ➔ Lab 4 python room1.py
Guessed pairs: {'T': 'E', 'E': 'T', 'R': 'R', 'Y': 'S', 'X': 'O', 'O': 'A', 'K': 'H', 'O': 'N', 'Z': 'D', 'P': 'I', 'I': 'P', 'G': 'S', 'S': 'L'}
THE SECRET MESSAGE IS HIDDEN IN THIS PARAGRAPH.
LETTER FREQUENCY DOES NOT REVEAL THE WHOLE KEY,
BUT IT GIVES A GOOD START.
LOOK FOR COMMON WORDS, REPEATED PATTERNS,
AND SHORT WORDS.
WHEN YOU RECOVER THE MESSAGE,
THE VIGENERE KEY HAS FOUR LETTERS.
```

3 Room 2

```
Problems Output Debug Console Postman Console Terminal Ports
(base) → Lab 4 python room2.py

Group 1
G 22.0 %
U 12.0 %
V 10.0 %
J 10.0 %
T 8.0 %
E 8.0 %
Q 6.0 %
R 4.0 %
P 4.0 %
C 4.0 %
O 4.0 %
M 4.0 %
X 2.0 %
W 2.0 %

Group 2
S 34.0 %
H 8.0 %
F 8.0 %
V 6.0 %
C 6.0 %
P 6.0 %
Q 4.0 %
I 4.0 %
A 4.0 %
G 4.0 %
Z 4.0 %
Y 2.0 %
T 2.0 %
M 2.0 %
E 2.0 %
B 2.0 %
O 2.0 %

Group 3
H 18.37 %
V 10.2 %
D 10.2 %
U 8.16 %
W 8.16 %
P 6.12 %
Q 6.12 %
Y 4.08 %
R 4.08 %
G 4.08 %
L 4.08 %
Z 2.04 %
O 2.04 %
N 2.04 %
I 2.04 %
X 2.04 %
K 2.04 %
B 2.04 %
F 2.04 %

Group 4
I 28.57 %
X 10.2 %
G 10.2 %
P 8.16 %
W 6.12 %
L 6.12 %
T 4.08 %
V 4.08 %
C 4.08 %
A 4.08 %
Q 2.04 %
F 2.04 %
H 2.04 %
S 2.04 %
Y 2.04 %
K 2.04 %
E 2.04 %
```

```
room2.py > ...
1 alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
2 text = {
3     "VVHMGQIIVYHRSUPGTWQJSPVPCWUPHGCIVVHPGHMTSDTRSDVUSYITMZL"
4     "GFHIXSUCVSDQSPFGVLIQIOHEVHMSDQJQRPMAQWGSXJSPSUHIVGEXIPHVC"
5     "OPRRCBGYUSLXVCUIECYITHKIMSBAJSQXJSPIUGDKPGHQAHMEZHETSQXGFWL"
6     "GDFGGVQAQFIEZLTUS"
7 }
8
9 key = ""
10 shifts = []
11
12 for i in range(4):
13     group = text[i::4]
14     counts = {}
15     for c in group:
16         counts[c] = counts.get(c, 0) + 1
17
18     print("\nGroup", i+1)
19     for c in sorted(counts, key=counts.get, reverse=True):
20         print(c, round(100*counts[c]/len(group), 2), "%")
21
22     frequent = max(counts, key=counts.get)
23     shift = (alphabet.index(frequent) - alphabet.index("E")) % 26
24     shifts.append(shift)
25     key += alphabet[shift]
26
27 print("\nKey:", key)
28
29 result = ""
30 for i in range(len(text)):
31     position = alphabet.index(text[i])
32     result += alphabet[(position-shifts[i % 4]) % 26]
33
34 print("Plaintext:", result)
```

```
Problems Output Debug Console Postman Console Terminal Ports
(base) → Lab 4 python room2.py
Y 2.04 %
K 2.04 %
E 2.04 %

Key: CODE
Plaintext: THESECRETKEEPERLEFTTHESENOTESBECAUSETHELETTEREAPPEARSEVERYWHEREEVERYTEAMMEMBERSHOULDCHECKEACHCOLUMN
SEEKTHEMOSTFREQUENTSMBOLANDUSEITTORECOVERTHEKEYWHENTHEMESSAGEBECOMESCLEARENTERTHEACCESSWORDECLIPSE
(base) → Lab 4
```

THE SECRET KEEPER LEFT THESE NOTES BECAUSE THE LETTER E APPEARS EVERYWHERE. EVERY TEAM MEMBER SHOULD CHECK EACH COLUMN. SEEK THE MOST FREQUENT SYMBOL AND USE IT TO RECOVER THE KEY. WHEN THE MESSAGE BECOMES CLEAR ENTER THE ACCESS WORD ECLIPSE.